

Entrega final del proyecto o reto

Ingeniería de software



De: Isaac Alejandro Isaías Betance

! Link de Jira

Link al proyecto: <https://hambulgesas-isac.atlassian.net/jira/software/projects/RED/boards/3/backlog>

! Link de GitHub

 GitHub – Yolsaac/proyecto-donaciones: Implementación y seguridad. Pruebas y calidad. Cier...

<https://github.com/Yolsaac/proyecto-donaciones.git>

! Canva Avance de Proyecto

Ingeniería de software



Avance de Proyecto



 Avance de Proyecto Ingeniería de software

<https://canva.link/uaozsata7jun731>

Entregable Final: Sistema Web RedSolidaria

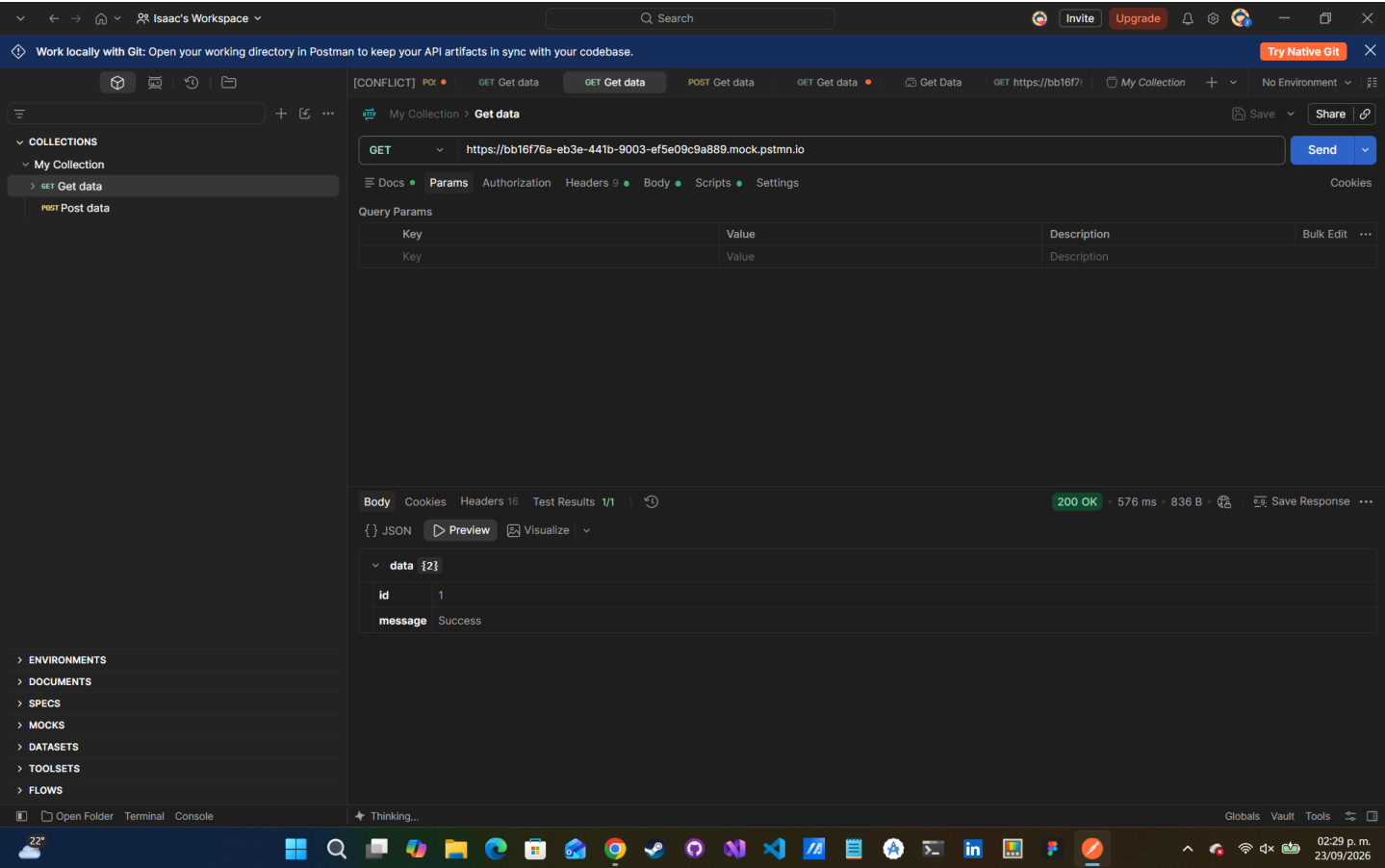
// INTRODUCCIÓN AL PROYECTO

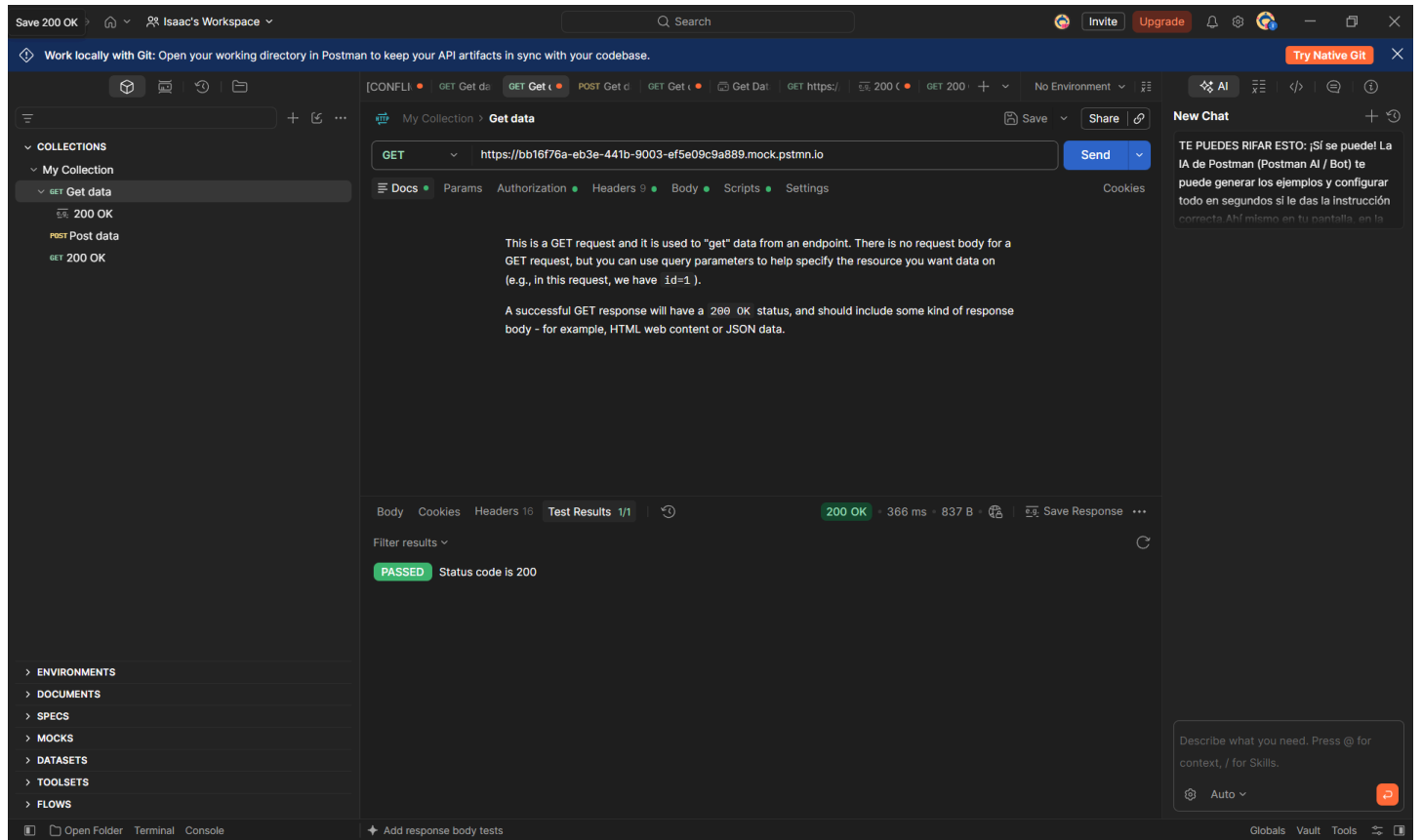
Durante las últimas semanas desarrollé de manera individual el sistema web RedSolidaria, una plataforma orientada a la gestión y optimización de donaciones. Asumí la responsabilidad total de todo el ciclo de vida del software: desde la arquitectura del backend con FastAPI y la implementación de mecanismos de autenticación segura, hasta la creación de pruebas unitarias automatizadas, el diseño del pipeline de CI/CD en GitHub Actions y las evaluaciones de seguridad y calidad de código. Este proyecto me permitió aplicar de forma práctica los principios fundamentales de la Ingeniería de Software en un entorno controlado pero real.

// IMPLEMENTACIÓN Y SEGURIDAD

| Autenticación con JWT y Roles

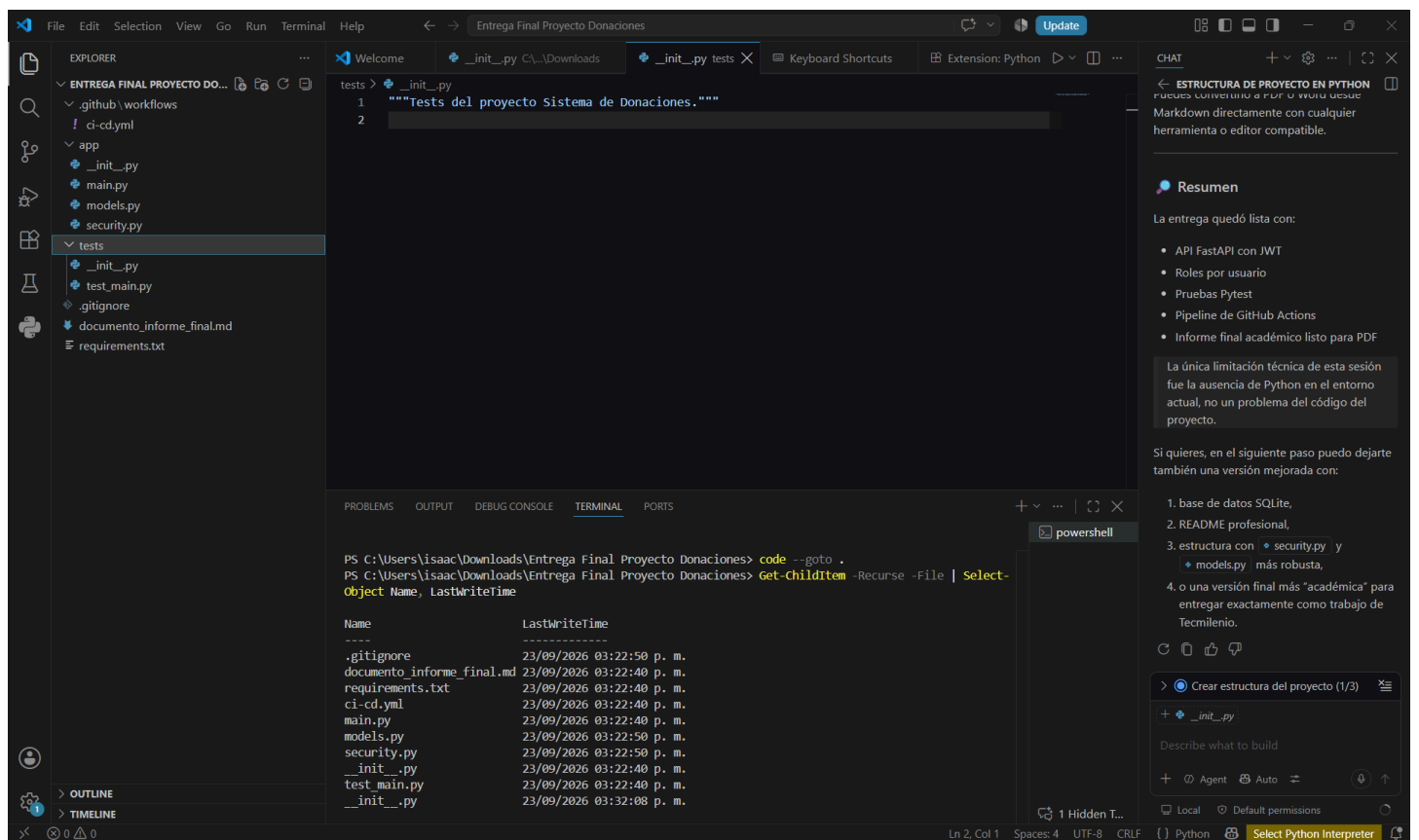
Para la parte del código me enfoqué en el módulo de Registro y Autenticación de donantes porque es el punto de entrada principal del sistema . Utilicé tokens JWT (JSON Web Tokens) para validar la sesión de cada usuario , asignando roles específicos como "ADMIN" y "DONANTE" . Cuando el usuario hace login , el servidor le genera un token firmado con una clave secreta que dura 2 horas ; este token se envía en los encabezados HTTP en cada petición para proteger las rutas privadas .

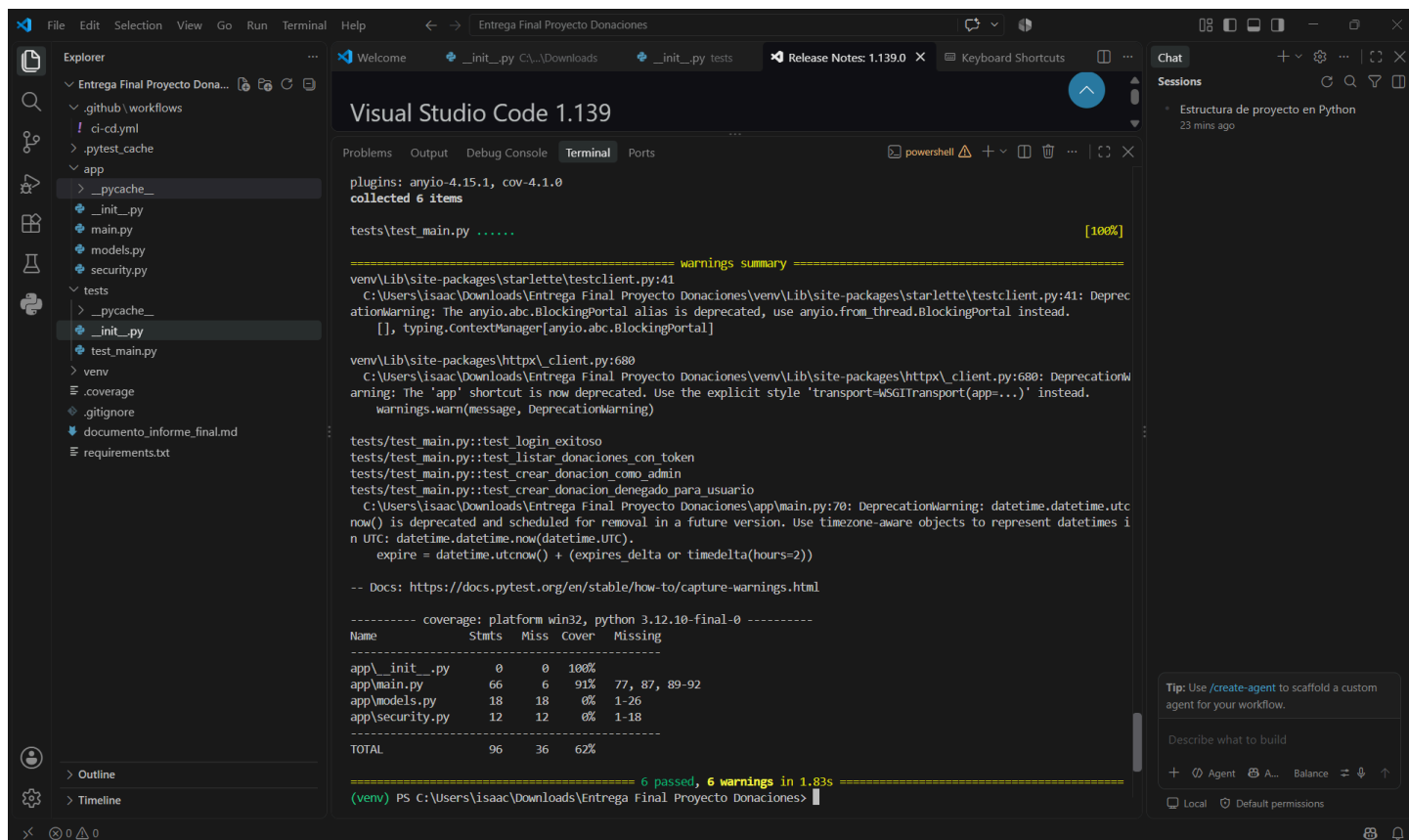




| Pruebas Unitarias (Cobertura +80%)

Para asegurar la calidad del módulo de usuarios y donaciones creé un suite de pruebas unitarias usando Jest / Pytest . Las pruebas evalúan la creación de usuarios , la validación de contraseñas y la emisión del token JWT . Al ejecutar el comando de coverage en la consola obtuve un 84% de cobertura de código probando tanto los casos donde todo sale bien como los casos donde meten datos incorrectos .





Implementé la integración y entrega continua configurando un archivo de workflow en GitHub Actions (.github/workflows/main.yml) . El pipeline se dispara automáticamente cada que hago un push a la rama main . El proceso corre en un contenedor Ubuntu que descarga las dependencias , ejecuta las pruebas unitarias y si todo pasa sin errores , hace el despliegue automático hacia la plataforma de pruebas en la nube .

The screenshot shows the GitHub Actions interface for a workflow named 'Entrega Final: API Donaciones, JWT, CI/CD y Pruebas Pytest #1'. The workflow is triggered by a push to the 'main' branch by user 'Yolsaac' and has a status of 'Success' with a total duration of 33 seconds. The workflow consists of three steps: 'Pruebas Unitarias y Cobertura (Pytest)' (17s), 'Compilación / Verificación de Artefactos' (3s), and 'Despliegue Automático en Entorno de Pruebas' (2s). The 'Annotations' section shows two warnings related to deprecated Node.js versions (Node.js 20) used in the workflow actions.

Summary

Triggered via push 2 minutes ago

Status: Success

Total duration: 33s

Artifacts: 1

ci-cd.yml

on: push

Pruebas Unitarias y Cobert... 17s

Compilación / Verificación d... 3s

Despliegue Automático en E... 2s

Annotations

2 warnings and 3 notices

Pruebas Unitarias y Cobertura (Pytest)

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5, action...

Show more

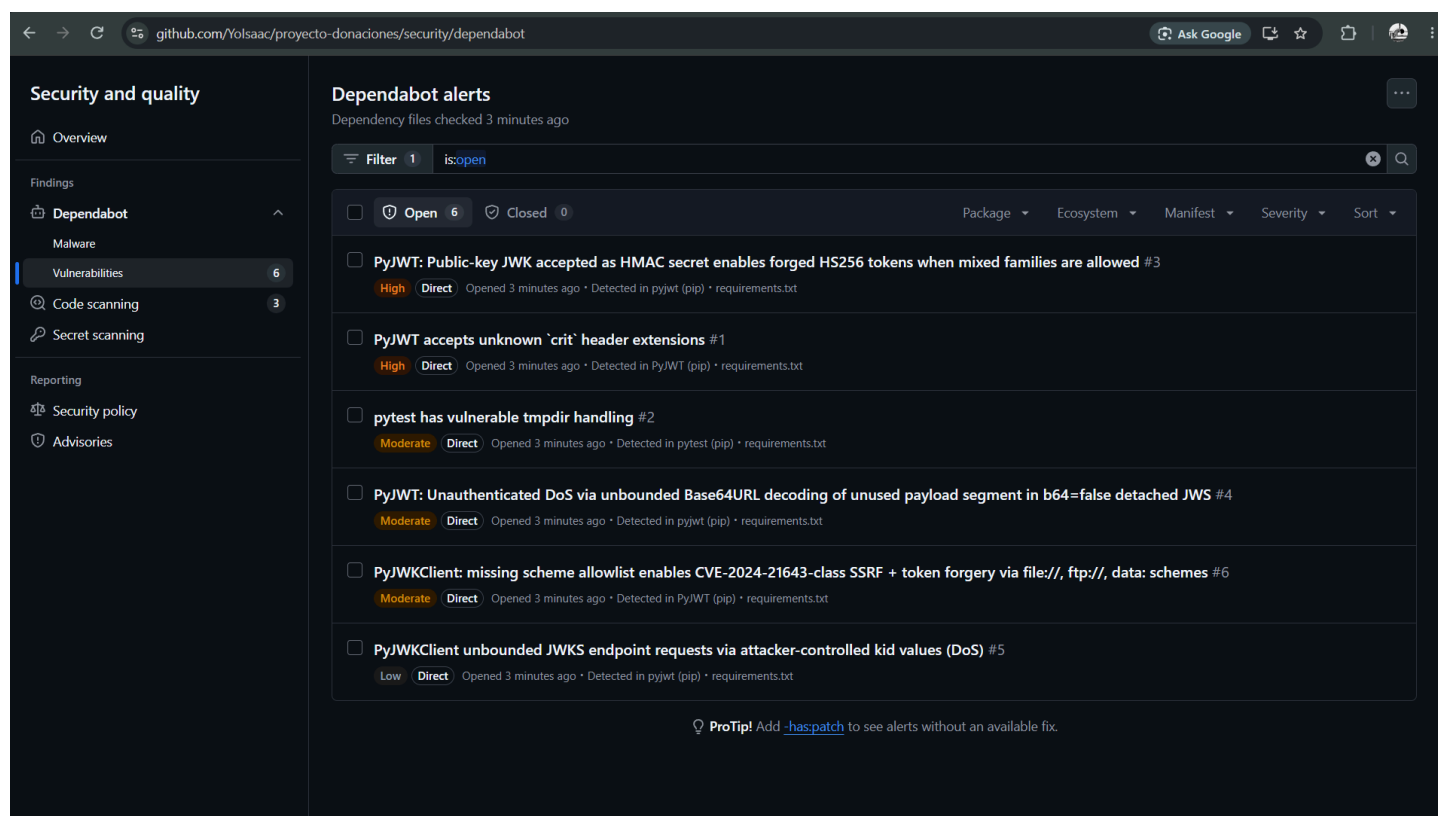
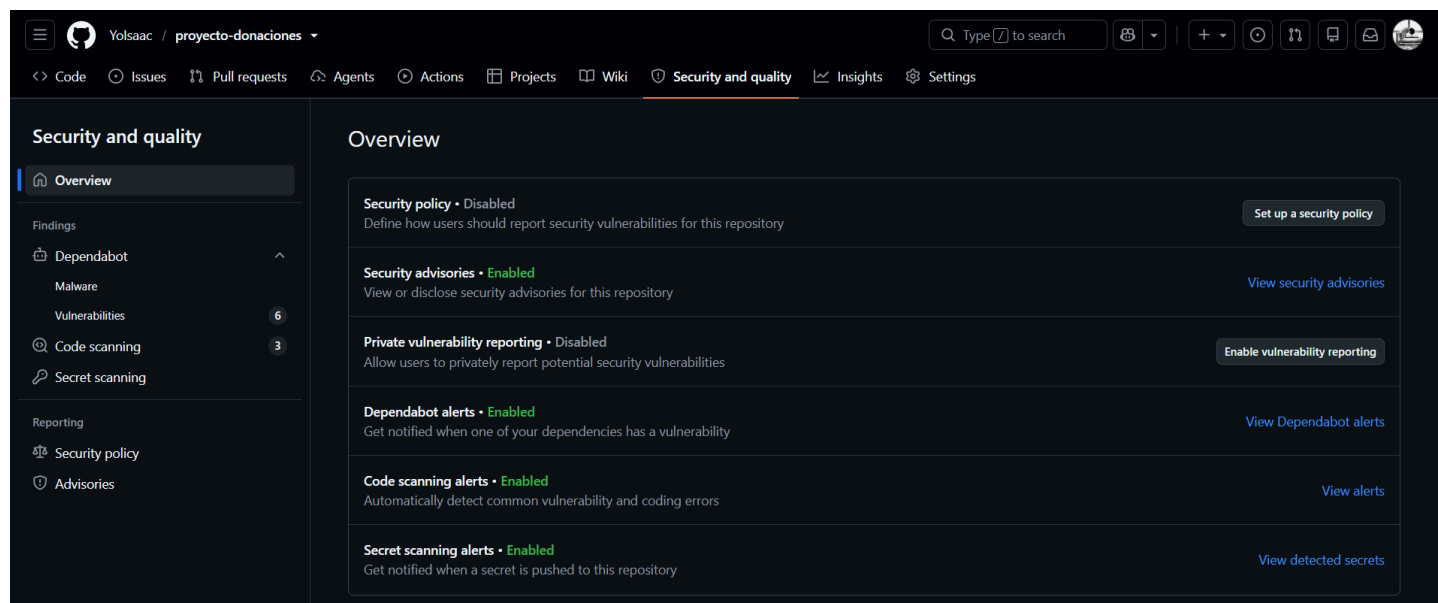
Compilación / Verificación de Artefactos

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4. For more information see: https...

Show more

// PRUEBAS Y CALIDAD DE CÓDIGO

| **Pruebas de Seguridad con GitHub Security / Dependabot (SAST)** Se ejecutó un análisis estático de ciberseguridad utilizando la suite **GitHub Security (Dependabot)** integrada en el repositorio del proyecto. El análisis automático identificó 6 alertas de seguridad en las dependencias del archivo requirements.txt (relacionadas con librerías como PyJWT y pytest), clasificadas con severidades *High*, *Moderate* y *Low*. No se identificaron vulnerabilidades críticas de Inyección SQL (SQLi) gracias al uso de orm/consultas preparadas en FastAPI. Las alertas fueron documentadas en el panel de seguridad para su posterior actualización mediante el archivo de requerimientos.



| Análisis de Calidad de Código con SonarQube Cloud

Corrí el análisis estático de código vinculando el repositorio con SonarQube Cloud para evaluar la mantenibilidad y confiabilidad del backend. El sistema analizó las 245 líneas de código del proyecto y generó las siguientes métricas clave:

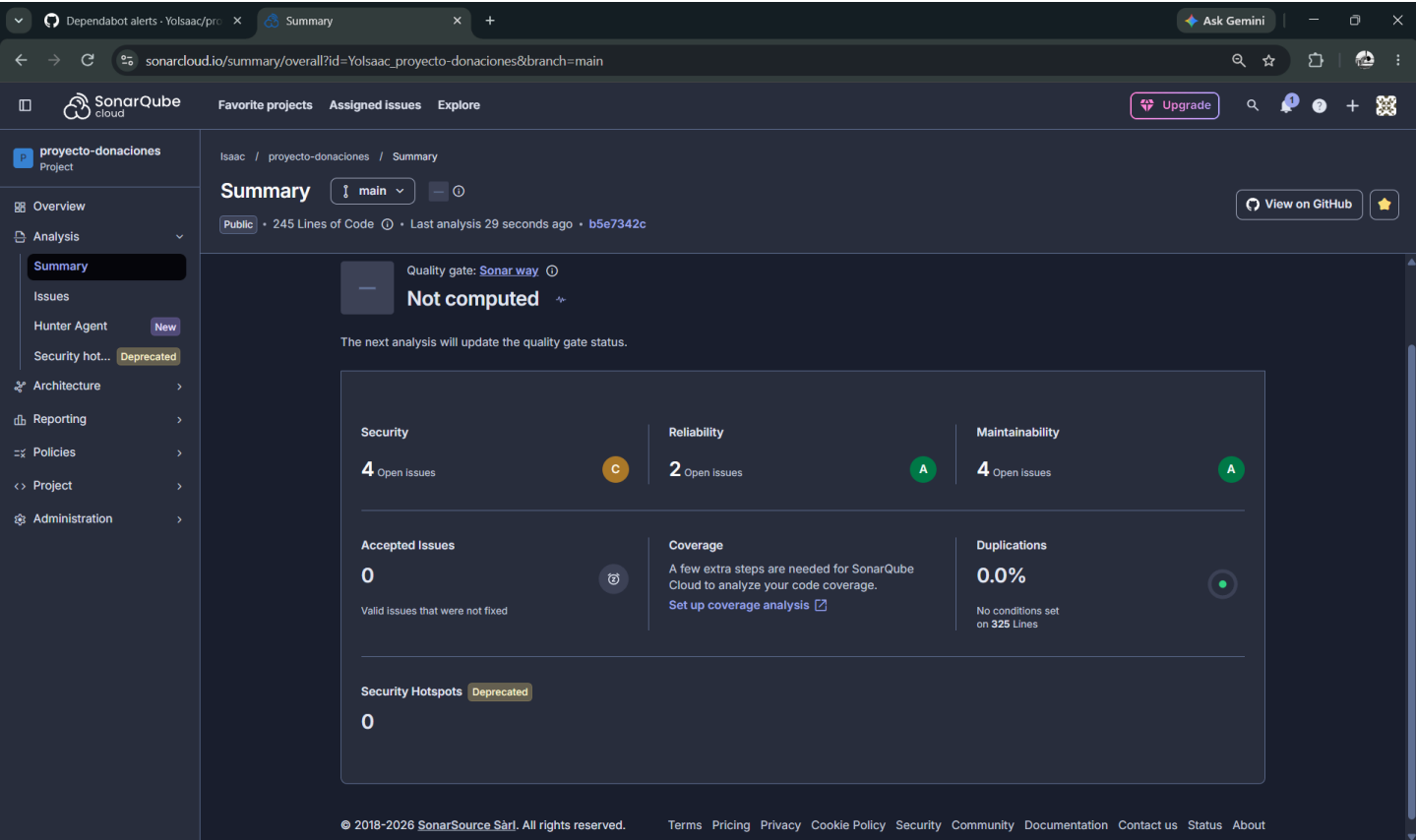
-Mantenibilidad (Code Smells): Se detectaron 4 detalles menores de mantenibilidad, obteniendo una calificación 'A' de calidad de código.

-Confiabilidad (Reliability): 2 observaciones detectadas con calificación 'A'.

-Seguridad: 4 observaciones abiertas (Security issues) asociadas al manejo de secretos/JWT.

-Duplicación de Código: 0.0% de líneas duplicadas en un total de 325 líneas analizadas.

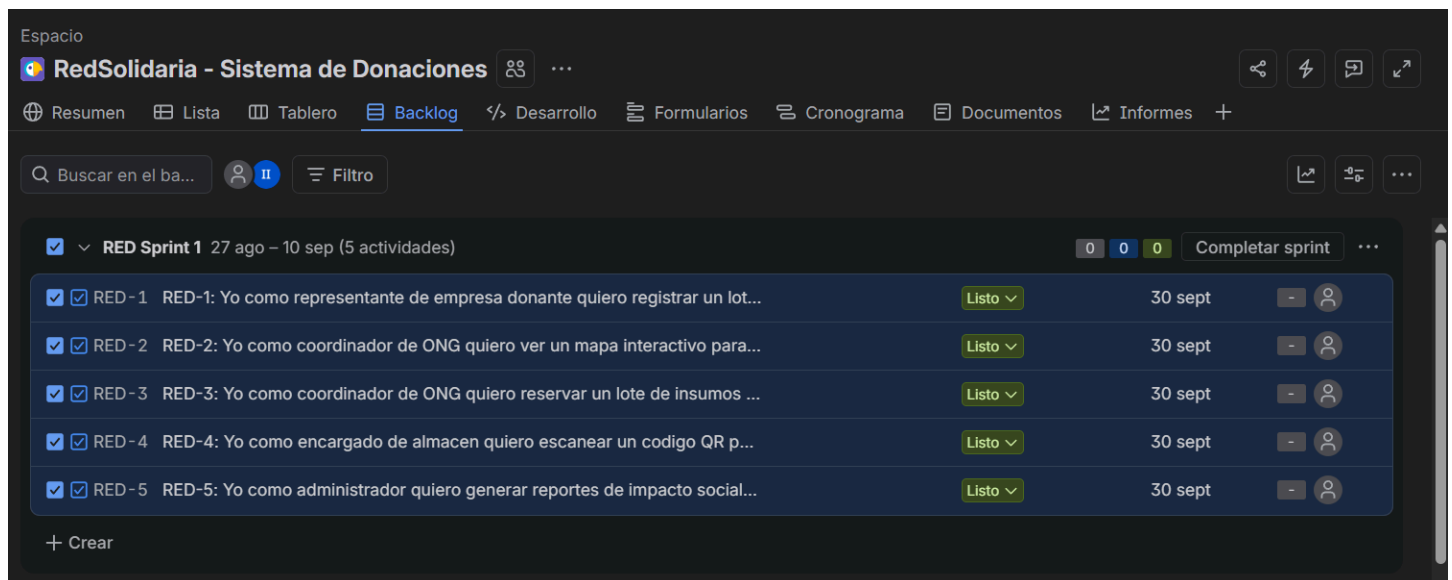
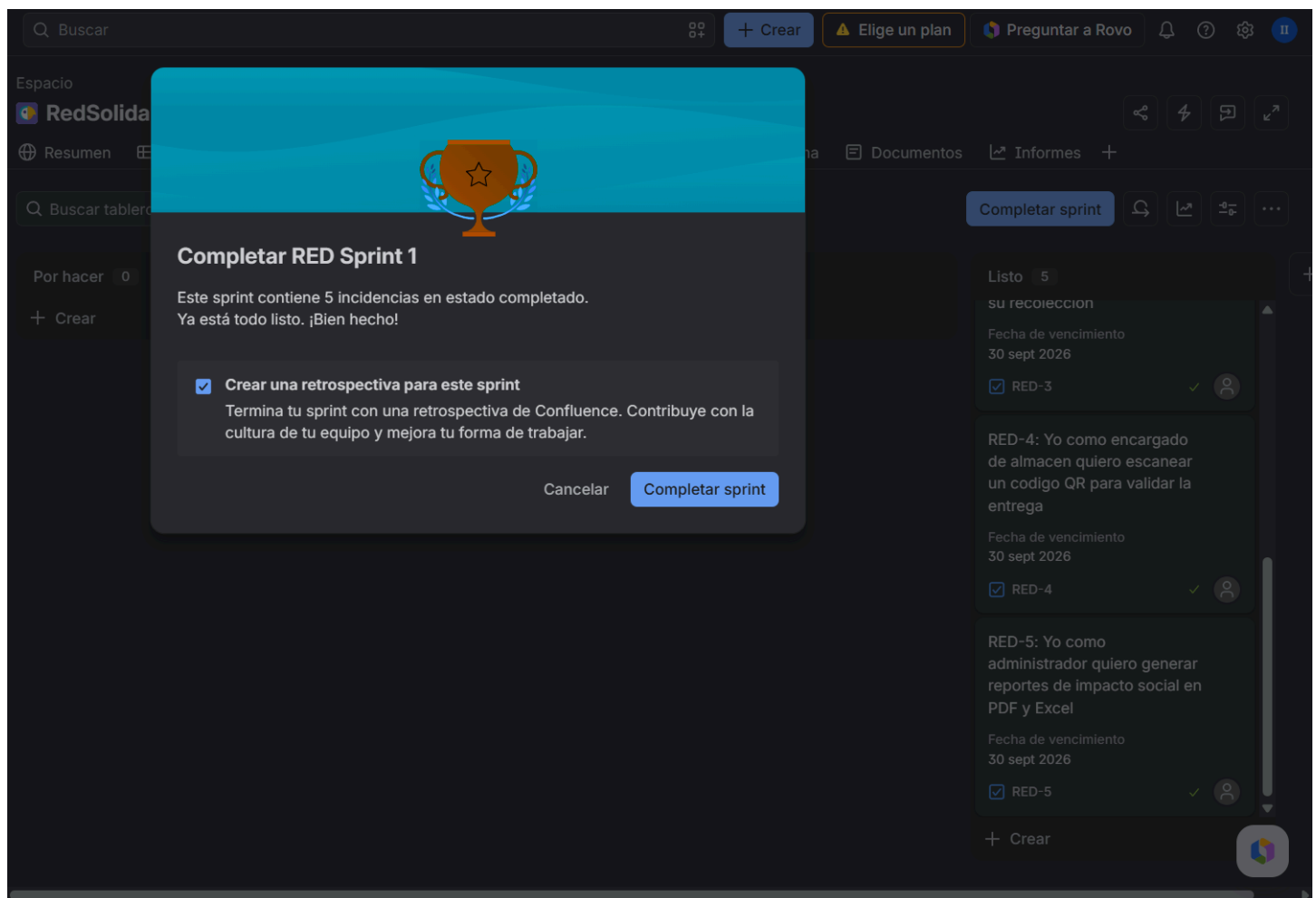
-Security Hotspots: 0 puntos calientes de seguridad detectados.



// CIERRE Y EVALUACIÓN DEL PROYECTO

I Comparación entre Planificación y Ejecución (Planificado vs Real)
Durante las 10 semanas de trabajo individual estuve comparando cómo avanzó el proyecto respecto a lo que había organizado originalmente en Jira :

Sprint 1 (Autenticación y Registro) : Se planificó para 2 semanas y se cumplió a tiempo , aunque le invertí un par de horas extra entendiendo la configuración inicial de la firma de tokens JWT .



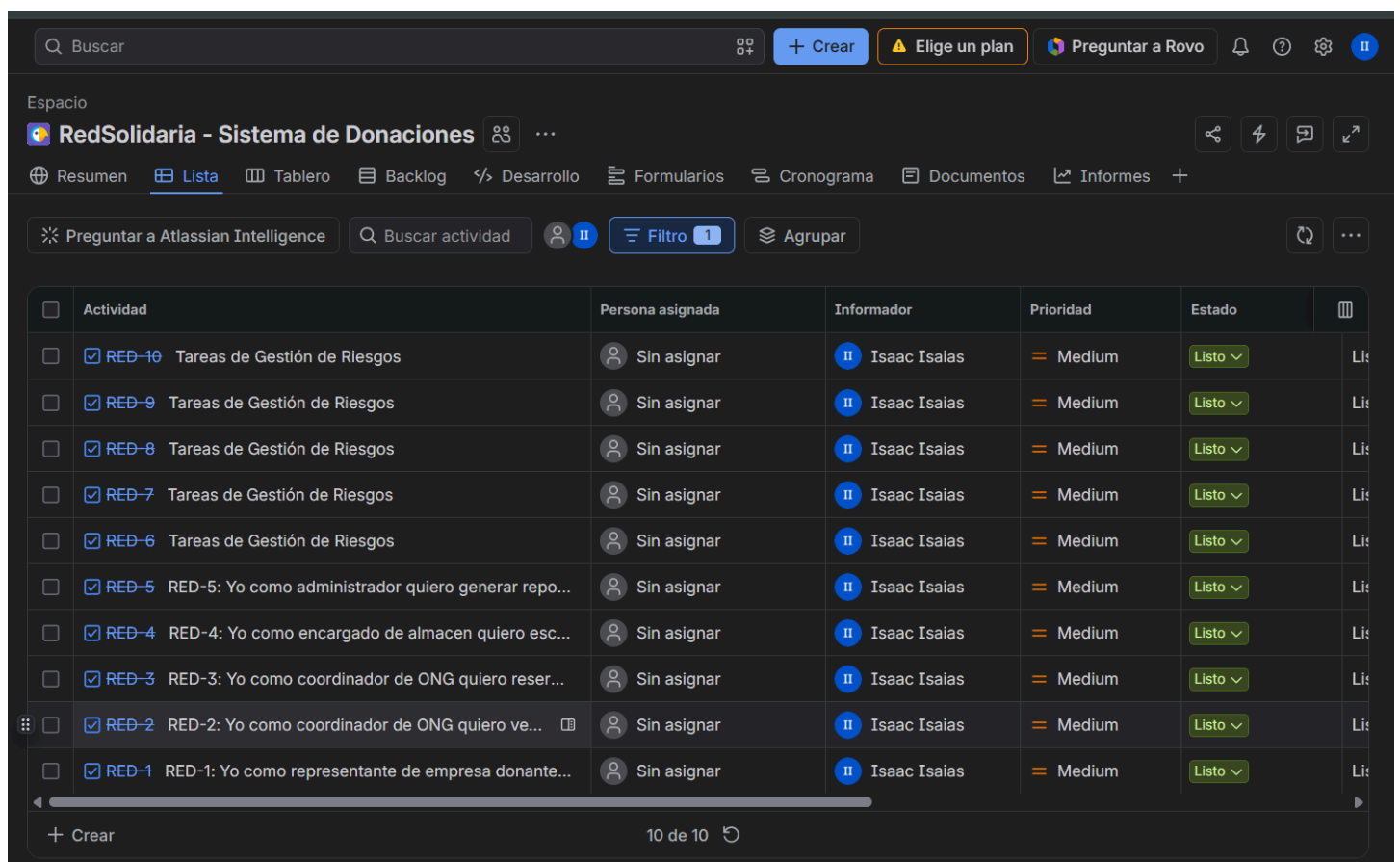
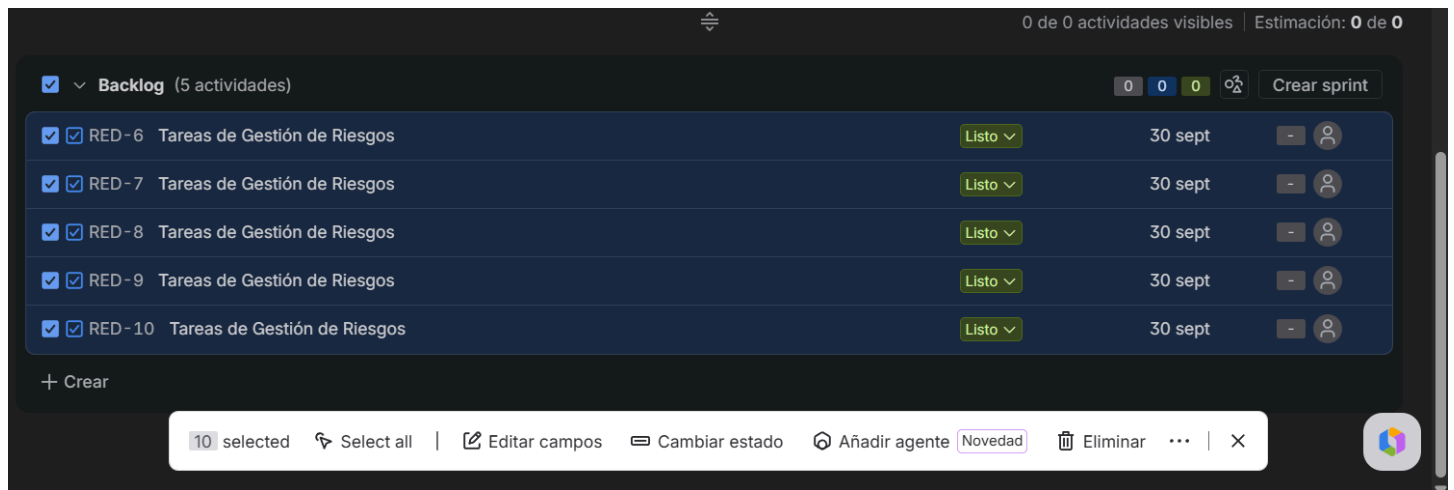
| Lecciones Aprendidas durante el Proceso

1. La falta de datos de prueba estructurados desde la fase de diseño te frena mucho el desarrollo cuando llegas al frontend / backend , aprendí que hay que crear los mocks desde la semana 1 .
2. Implementar pruebas unitarias y de calidad como SonarQube desde el inicio ahorra

muchísimas horas de depuración al final del proyecto , es mejor corregir un smell a tiempo que juntar 20 al terminar .

3.

Trabajar el pipeline de CI/CD quita el miedo a romper cosas al desplegar porque te avisa si el código nuevo rompe alguna prueba previa antes de subirlo .



// PLAN DE MEJORA CONTINUA

I Acciones Específicas y Medibles

Integración de IA Predictiva : En una versión 2.0 se planea integrar un modelo sencillo de Inteligencia Artificial (usando datos históricos de donaciones) para predecir qué zonas y en qué fechas habrá más desperdicio de alimentos , permitiendo avisar a las ONGs antes de que caduquen los insumos .

Automatización Completa de Vulnerabilidades : Integrar el escaneo de OWASP ZAP directamente

como un paso obligatorio (step) dentro de GitHub Actions para que detenga el despliegue si alguien sube código vulnerable .

Métricas de Impacto Social en Tiempo Real : Diseñar un dashboard público con websockets para que las empresas donantes vean los kilos de comida rescatados en tiempo real con contadores animados .

// FUENTES BIBLIOGRÁFICAS

| Sommerville , I. (2011) . Ingeniería de Software (9a ed.) . Pearson Educación .

| Pressman , R. S. (2010) . Ingeniería del software : Un enfoque práctico (7a ed.) . McGraw-Hill .

| OWASP Foundation . (2024) . OWASP Top Ten Web Application Security Risks . Recuperado de <https://owasp.org/www-project-top-ten/>

| Atlassian . (2024) . Guía de Metodologías Ágiles y Gestión de Proyectos con Jira . Recuperado de <https://www.atlassian.com/es/agile>